

# Data Lake Implementation using AWS Glue and Athena

**Services:** Amazon S3, AWS Glue, Amazon Athena, IAM

**Description:** Create a serverless data lake using Amazon S3 as the storage layer. Use AWS Glue Crawlers to automatically discover and catalog data stored in S3 and store metadata in the Glue Data Catalog. Query the data directly using Amazon Athena without managing any servers. Configure IAM roles and policies to ensure secure access to data and services.

**Skills:** Data Lake Architecture, Data Cataloging, Serverless Analytics, AWS Glue Crawlers, Amazon Athena Querying, S3 Data Storage, IAM Access Management.

## Definition:

The Data Lake Implementation using AWS Glue and Athena is a serverless data analytics solution that stores structured and semi-structured data in Amazon S3. AWS Glue automatically discovers and catalogs datasets, while Amazon Athena enables SQL-based analysis directly on data stored in S3 without requiring database servers.

## Scope of the Project:

The scope of this project is to design and implement a fully serverless Data Lake architecture using AWS services. The project demonstrates how organizations can store, catalog, secure, and analyze large volumes of data without managing servers or databases.

## The project covers:

- Data Storage using Amazon S3: Create a centralized Data Lake using Amazon S3.
- Data Cataloging using AWS Glue: Automatically discover datasets stored in S3.
- Serverless Querying using Athena: Query data directly from Amazon S3 using SQL.
- Data Discovery and Metadata Management: Create databases and tables in Glue Data Catalog. Maintain a centralized metadata repository.
- Security and Access Control using IAM: Implement IAM Roles and Policies.
- Cost Optimization using serverless services: Use fully managed serverless services.

## Methodology:

### 1. S3 + Glue + Athena (Best Method)

- Fully Serverless Architecture
- No Database Administration Required
- Automatic Schema Discovery
- Highly Scalable
- Cost Effective
- Easy to Implement

## 2. Traditional Database Approach (Alternative Method)

### EC2+MySQL/PostgreSQL + Manual ETL Scripts

- Requires server management.
- Higher operational costs.
- Manual schema management.
- Limited scalability compared to serverless architecture.

### AWS Services Used:

#### 1. Amazon S3 (Simple Storage Service)

**Definition:** Amazon S3 is an object storage service that provides scalable, durable, and secure storage.

**Purpose:** Stores raw datasets and Athena query results.

**Role:** Acts as the storage layer of the Data Lake.

#### 2. AWS Glue

**Definition:** AWS Glue is a fully managed data integration and metadata cataloging service.

**Purpose:** Discovers and catalogs datasets stored in S3.

**Role:** Creates metadata tables using Crawlers and stores them in Glue Data Catalog.

#### 3. Amazon Athena

**Definition:** Amazon Athena is a serverless interactive query service that allows SQL queries directly on S3 data.

**Purpose:** Perform analytics on stored datasets.

**Role:** Executes SQL queries using metadata from Glue Catalog.

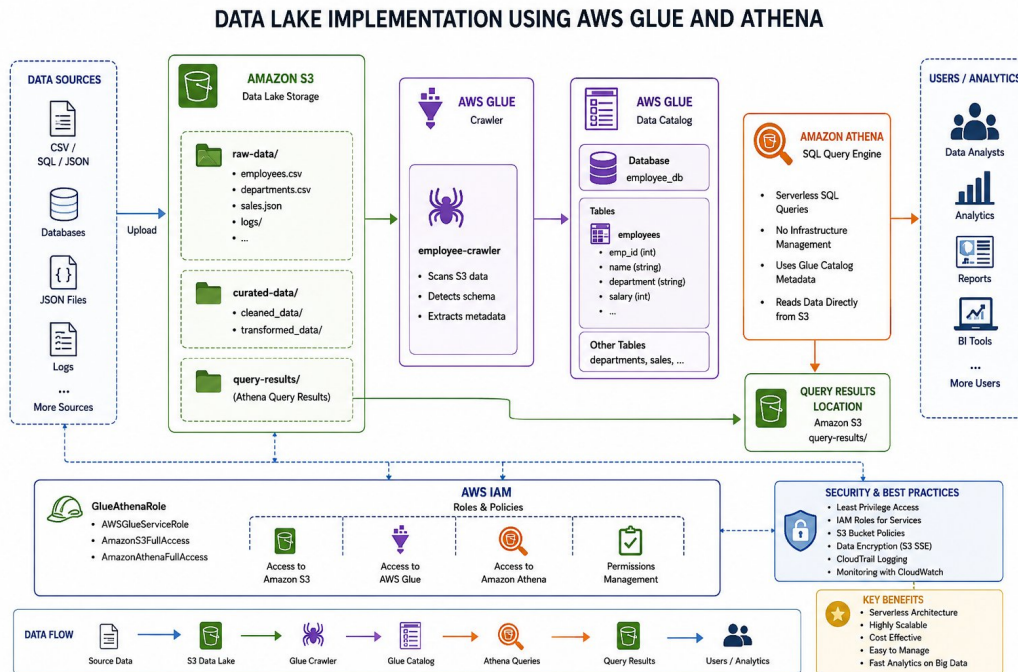
#### 4. AWS IAM:

**Definition:** AWS Identity and Access Management (IAM) controls access to AWS resources.

**Purpose:** Provides secure access management.

**Role:** Creates roles and policies for Glue and Athena operations.

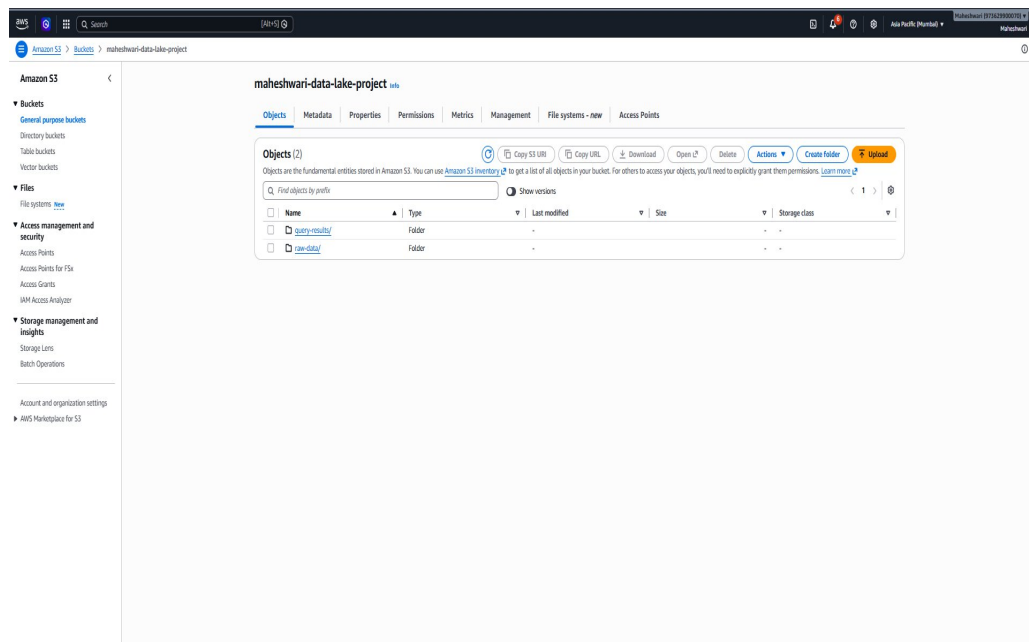
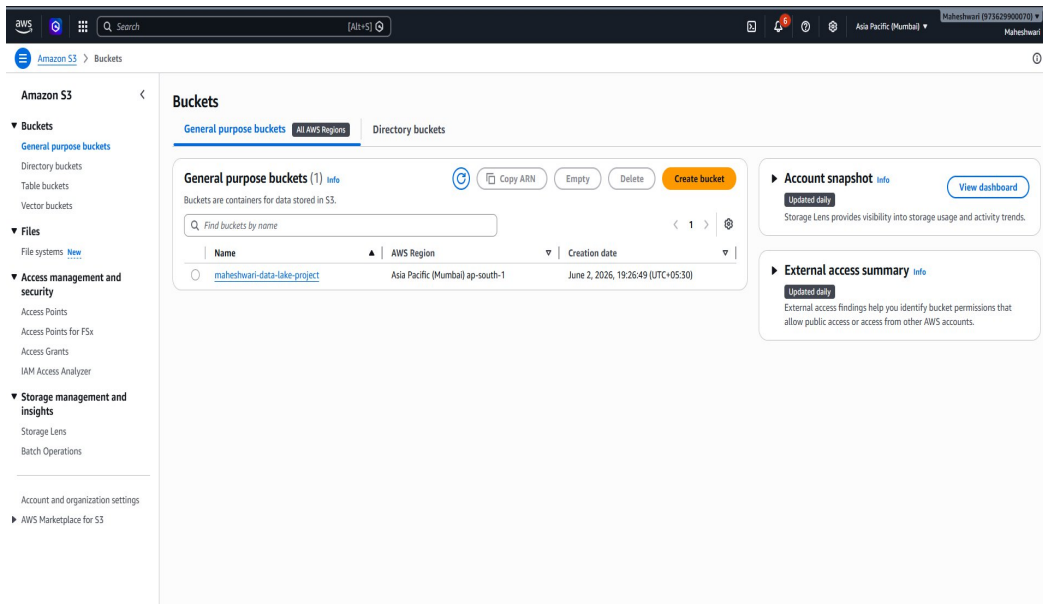
## Architecture:



## Implementation Steps:

### Step 1: Create S3 Bucket and folders (raw-data, query-results)

- Login to AWS Console.
- Open S3 service.
- Click Create Bucket.
- Bucket Name: Maheshwari-data-lake-project
- Select Region: ap-south-1 (Mumbai).
- Keep defaults and create bucket.
- Create folders:
  - raw-data/
  - query-results/



## Step 2: Upload employees.csv dataset

Add Employee's details

emp\_id, name, department, salary

101, Ravi, IT,50000

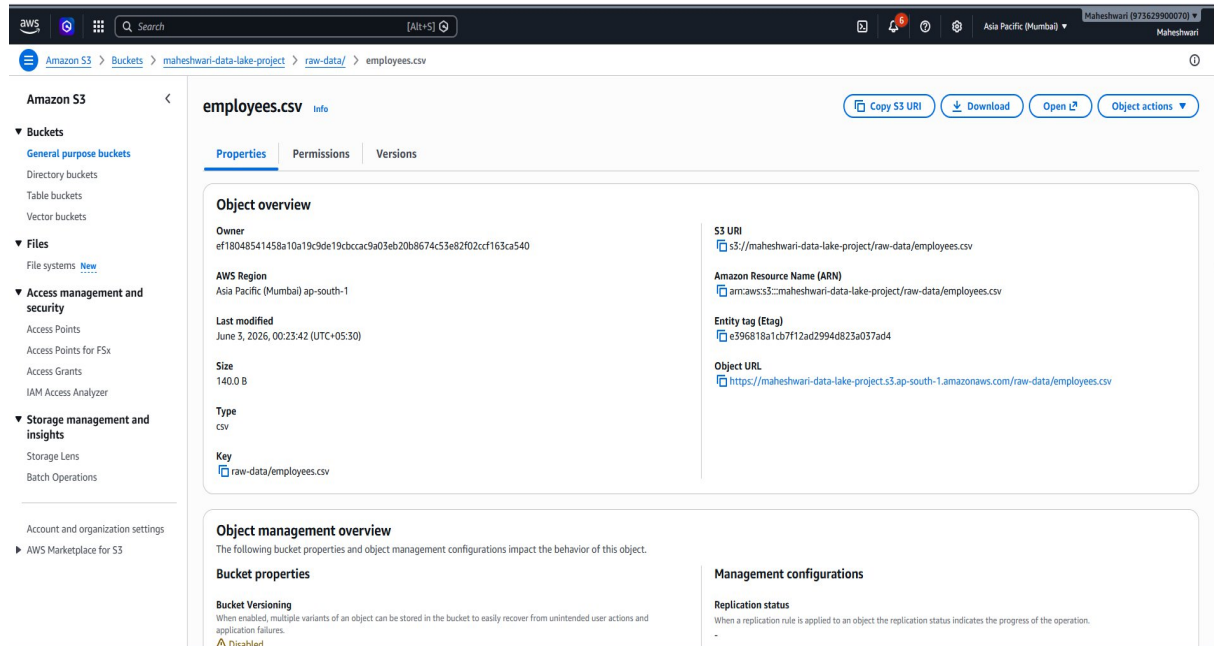
102, Priya, HR,45000

103, Arjun, Finance,60000

104, Kiran, IT,55000

105, Meena, Marketing,48000

1. Open raw-data folder.
2. Click Upload.
3. Upload employees.csv.
4. Verify file exists in S3.



### Step 3: Create IAM Role (GlueAthenaRole)

- IAM → Roles → Create Role.
- Trusted Entity: AWS Service.
- Use Case: Glue.
- Attach:
  - AWSGlueServiceRole
  - AmazonS3FullAccess
- Role Name: GlueAthenaRole.
- Create Role.

aws IAM Roles GlueAthenaRole

### GlueAthenaRole Info

Allows Glue to call AWS services on your behalf.

[Delete](#)

#### Summary Edit

|                                                          |                                                                             |
|----------------------------------------------------------|-----------------------------------------------------------------------------|
| <b>Creation date</b><br>June 02, 2026, 19:35 (UTC+05:30) | <b>ARN</b><br><a href="#">arn:aws:iam::973629900070:role/GlueAthenaRole</a> |
| <b>Last activity</b><br><span>✓ 14 hours ago</span>      | <b>Maximum session duration</b><br>1 hour                                   |

[Permissions](#) | [Trust relationships](#) | [Tags](#) | [Last Accessed](#) | [Revoke sessions](#)

#### Permissions policies (2) Info

You can attach up to 10 managed policies.

[Simulate](#) [Remove](#) [Add permissions](#)

Filter by Type: [All types](#)

## Step 4: Create Glue Database (employee\_db)

- Open AWS Glue.
- Data Catalog → Databases.
- Create Database.
- Name: employee\_db.
- Save.

aws AWS Glue Databases employee\_db

### employee\_db

Last updated (UTC) June 2, 2026 at 19:08:32 [Edit](#) [Delete](#)

#### Database properties

| Name        | Description | Location | Created on (UTC)         |
|-------------|-------------|----------|--------------------------|
| employee_db | -           | -        | June 2, 2026 at 17:45:02 |

#### Tables (1)

Last updated (UTC) June 2, 2026 at 19:08:34 [Delete](#) [Add tables using crawler](#) [Add table](#)

View and manage all available tables.

| <input type="checkbox"/> | Name                     | Database    | Location                  | Classification | Deprecated | View data                  | Data quality                      | Column statistics               |
|--------------------------|--------------------------|-------------|---------------------------|----------------|------------|----------------------------|-----------------------------------|---------------------------------|
| <input type="checkbox"/> | maheshwari_data_lake_pro | employee_db | s3://maheshwari-data-lake | CSV            | -          | <a href="#">Table data</a> | <a href="#">View data quality</a> | <a href="#">View statistics</a> |

## Step 5: Create Glue Crawler (employee-crawler)

Glue → Crawlers → Create Crawler.

2. Name: employee-crawler.

3. Data Source: S3.

4. Path: s3://maheshwari-data-lake-project/raw-data/

5. IAM Role: GlueAthenaRole.

6. Target Database: employee\_db.

7. Create Crawler.

The screenshot displays the AWS Glue console interface for the 'employee-crawler'. The left sidebar shows the navigation menu with categories like AWS Glue, Data Catalog, Data Integration and ETL, and Legacy pages. The main content area is titled 'employee-crawler' and includes a 'Crawler properties' section with fields for Name, Description, Maximum table threshold, IAM role, Security configuration, Database, Lake Formation configuration, State, and Table prefix. Below this is an 'Advanced settings' section. The 'Crawler runs' tab is active, showing a table of crawler runs. The table has columns for Start time (UTC), End time (UTC), Current/last duration, Status, DPU hours, and Table changes. A single run is listed with a status of 'Completed'.

| Start time (UTC)         | End time (UTC)           | Current/last duration | Status    | DPU hours | Table changes                       |
|--------------------------|--------------------------|-----------------------|-----------|-----------|-------------------------------------|
| June 2, 2026 at 18:14:23 | June 2, 2026 at 18:15:04 | 40 s                  | Completed | 0.114     | 1 table change, 2 partition changes |

## Step 6: Run Crawler and verify employee's table

Select employee-crawler.

2. Click Run.

3. Wait until status becomes Succeeded.

4. Verify employees table is created.

aws [Search] [Alt+S] Asia Pacific (Mumbai) Maheshwari (973629900070) Maheshwari

AWS Glue > Tables > maheshwari\_data\_lake\_project

**AWS Glue**

- Getting started
- ETL jobs
- Visual ETL
- Notebooks
- Job run monitoring
- Data Catalog tables
- Data connections
- Workflows (orchestration)
- Zero-ETL integrations [New](#)

▼ **Data Catalog**

- Catalogs
- Databases
- Tables**
- Stream schema registries
- Schemas
- Connections
- Crawlers
- Classifiers
- Catalog settings

► **Data Integration and ETL**

► **Legacy pages**

- What's New [i](#)
- Documentation [i](#)
- AWS Marketplace

☐ Enable compact mode

☐ Enable new navigation

**Table details**

**Name**  
maheshwari\_data\_lake\_project

**Database**  
[employee\\_db](#)

**Description**  
-

**Last updated**  
June 2, 2026 at 18:15:03

**Classification**  
CSV

**Location**  
[s3://maheshwari-data-lake-project/](#)

**Connection**  
-

**Deprecated**  
-

**Column statistics**  
No statistics

► **Advanced properties**

**Schema** | Partitions | Indexes | Column statistics - new

**Schema (5)**

View and manage the table schema.

| # | Column name | Data type | Partition key | Comment |
|---|-------------|-----------|---------------|---------|
| 1 | emp_id      | bigint    | -             | -       |
| 2 | name        | string    | -             | -       |
| 3 | department  | string    | -             | -       |
| 4 | salary      | bigint    | -             | -       |
| 5 | partition_0 | string    | Partition (0) | -       |

[Edit schema as JSON](#) [Edit schema](#)

aws [Search] [Alt+S] Asia Pacific (Mumbai) Maheshwari (973629900070) Maheshwari

AWS Glue > Crawlers > employee-crawler

**AWS Glue**

- Getting started
- ETL jobs
- Visual ETL
- Notebooks
- Job run monitoring
- Data Catalog tables
- Data connections
- Workflows (orchestration)
- Zero-ETL integrations [New](#)

▼ **Data Catalog**

- Catalogs
- Databases
- Tables
- Stream schema registries
- Schemas
- Connections
- Crawlers**
- Classifiers
- Catalog settings

► **Data Integration and ETL**

► **Legacy pages**

- What's New [i](#)
- Documentation [i](#)
- AWS Marketplace

☐ Enable compact mode

☐ Enable new navigation

**employee-crawler**

Last updated (UTC)  
June 3, 2026 at 09:15:52

[Run crawler](#) [Edit](#) [Delete](#)

**Crawler properties**

**Name**  
employee-crawler

**IAM role**  
[GlueAthenaRole](#)

**Database**  
employee\_db

**State**  
READY

**Description**  
to test

**Security configuration**  
-

**Lake Formation configuration**  
-

**Table prefix**  
-

**Maximum table threshold**  
-

► **Advanced settings**

**Crawler runs** | Schedule | Data sources | Classifiers | Tags

**Crawler runs (1)**

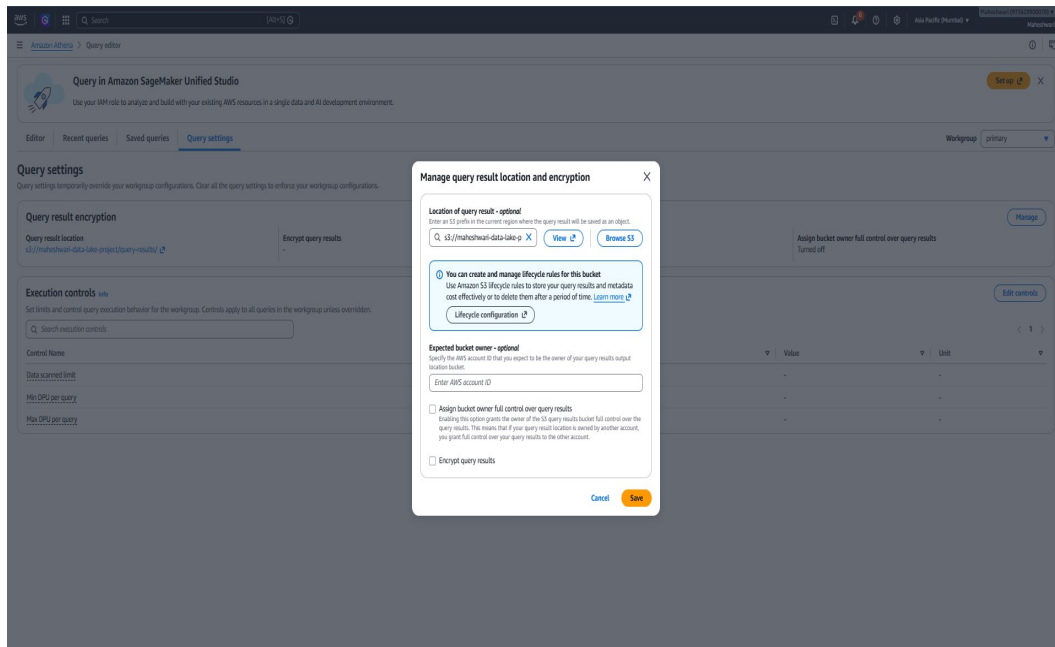
The list of crawler runs for this crawler.

| Start time (UTC)         | End time (UTC)           | Current/last duration | Status    | DPU hours | Table changes                       |
|--------------------------|--------------------------|-----------------------|-----------|-----------|-------------------------------------|
| June 2, 2026 at 18:14:23 | June 2, 2026 at 18:15:04 | 40 s                  | Completed | 0.114     | 1 table change, 2 partition changes |

[Stop run](#) [View CloudWatch logs](#) [View run details](#)

## Step 7: Configure Athena output location

- Open Athena Query Editor.
- Open Settings.
- Configure Query Result Location: `s3://maheshwari-data-lake-project/query-results/`
- Save.



## Step 8: Query data using SQL and see output

### Run Queries

1)SELECT department,

AVG (salary)

FROM maheshwari-data-lake-project

GROUP BY department;

2) SELECT \*

FROM maheshwari-data-lake-project

WHERE department='IT';

**Query 1 :**

```

1 SELECT department,
2 AVG(salary)
3 FROM maheshwari_data_lake_project
4 GROUP BY department;
5

```

**Query results** | Query stats

Completed Time in queue: 51 ms Run time: 575 ms Data scanned: 1.18 KB

**Results (9)**

| # | department  | salary  |
|---|-------------|---------|
| 1 | Finance     | 60000.0 |
| 2 | "IT"        |         |
| 3 | "Finance"   |         |
| 4 | Marketing   | 48000.0 |
| 5 | "Marketing" |         |
| 6 |             |         |
| 7 | HR          | 45000.0 |
| 8 | IT          | 52500.0 |
| 9 | "HR"        |         |

**Query 1 :**

```

1 SELECT *
2 FROM maheshwari_data_lake_project
3 WHERE department = 'IT';
4

```

**Query results** | Query stats

Completed Time in queue: 95 ms Run time: 524 ms Data scanned: 1.44 KB

**Results (2)**

| # | emp_id | name  | department | salary | partition_0 |
|---|--------|-------|------------|--------|-------------|
| 1 | 101    | Ravi  | IT         | 50000  | raw-data    |
| 2 | 104    | Kiran | IT         | 55000  | raw-data    |

## Step 9: Configure IAM security policies

Create policy allowing S3 read access to the bucket and attach it to required user's roles.

Create User → User1 and attach permissions.

The screenshot shows the AWS IAM console interface. The left sidebar contains the 'Identity and Access Management (IAM)' menu with options like Dashboard, Access Management, and Access reports. The main content area displays the 'DataLakeS3ReadOnlyPolicy' details. The 'Policy details' section shows the policy type as 'Customer managed', creation and edited times as 'June 03, 2026, 00:18 (UTC+05:30)', and the ARN as 'arn:aws:iam::973629900070:policy/DataLakeS3ReadOnl yPolicy'. Below this, the 'Permissions' tab is selected, showing 'Permissions defined in this policy' with a search bar and a toggle for 'Show remaining 468 services'.

The screenshot shows the AWS IAM console interface for the 'user1' user. The left sidebar contains the 'Identity and Access Management (IAM)' menu. The main content area displays the 'user1' details. The 'Summary' section shows the ARN as 'arn:aws:iam::973629900070:user/user1', console access as 'Enabled without MFA', and the last console sign-in as 'Never'. Below this, the 'Permissions' tab is selected, showing 'Permissions policies (1)' with a search bar and a 'Filter by Type' dropdown set to 'All types'.

## Troubleshooting:

### Issue 1: Crawler Failed

**Solution:** Verify S3 path and IAM permissions.

### Issue 2: Athena Query Failed

**Solution:** Configure Athena output location.

### Issue 3: Table Not Created

**Solution:** Verify CSV headers and rerun crawler.

### Issue 4: Access Denied

**Solution:** Attach Glue, S3 and Athena permissions.

**Conclusion:**

The Data Lake Implementation using AWS Glue and Athena provides a scalable, secure, and cost-effective serverless analytics platform. Amazon S3 serves as the storage layer, AWS Glue automates metadata discovery, and Athena enables SQL-based querying without managing servers.